

theme2-interop Solution Document

Theme 2: Two-Way Interoperability between SWS and Department Systems

PAN IIT “AI for Bharat” Hackathon — Solution Document Theme Sponsor: Department of Commerce & Industry, Government of Karnataka **Team:** AI for Bharat | April 2026

1. Executive Summary

Karnataka’s Single Window System (SWS) was designed to be the single front door for business registrations. In practice, 40+ legacy department systems continue to operate independently. A business that updates its address on SWS has no guarantee that the Labour Department, KSPCB, or Commercial Tax will ever see that change — and vice versa. This is the classic **split-brain problem**: two authoritative sources of truth that silently drift apart, creating regulatory ambiguity for lakhs of businesses and thousands of officers.

We propose **InteropSync** — a lightweight, event-sourced middleware that sits *alongside* both SWS and department systems without modifying either. It propagates changes bidirectionally, translates between incompatible schemas, detects conflicting data through fuzzy matching, and maintains a complete audit trail of every propagation. The Unified Business Identifier (UBID) from our Theme 1 submission serves as the join key that links a business across all systems.

Our working prototype covers 5 departments (Labour, KSPCB, Commercial Tax, Factories, Fire Safety) with per-department schema translation, idempotent sync operations, and a conflict resolution dashboard. The architecture is designed for incremental rollout — start with 3 departments in a pilot, then extend to all 40+ without architectural changes.

Key differentiator: We have actually built Theme 1’s entity resolution engine. Most competitors will mock the UBID dependency. We deliver a working cross-theme integration where a resolved UBID flows directly into the interoperability layer.

2. Problem Deep Dive

2.1 The Split-Brain Problem in Government Systems

When Karnataka launched SWS, the intent was clear: one portal for all business approvals. However, the 40+ department systems — Labour, KSPCB, Commercial Tax, Factories & Boilers, Fire & Emergency Services, and dozens more — were never decommissioned. Each department continues to accept applications, update statuses, and issue approvals through its own legacy system.

Consider a concrete scenario. Rajesh Kumar registers “Kumar Engineering Works” on SWS. The Labour Department creates its own record, calling it an “establishment” with its own field names, status codes, and reference numbers. Three months later, Rajesh changes his registered address on SWS. Labour never learns of this. Meanwhile, the Labour inspector updates the status to “Registered” — but SWS still shows “Pending Department Approval.”

Both systems now hold stale data. The result: compliance certificates issued to old addresses, tax notices sent to wrong premises, and officers making decisions on outdated information.

2.2 Why This Problem Persists

No common identifier. Before UBID (Theme 1), there was no reliable way to match “SWS Application #SWS-2024-KA-78432” to “Labour Registration #LBR/BNG/2024/11089.” Systems use PAN, GSTIN, shop licence numbers, or proprietor names — none universal or consistent.

Incompatible schemas. SWS calls it “business_name”; Labour calls it “establishment_name”; KSPCB calls it “unit_name.” SWS says “approved”; Labour says “Registered”; KSPCB says “CTO Granted.” These reflect different conceptual models of what a “business” is to each department.

No event infrastructure. Most department systems were built in the 2000s-2010s without events, webhooks, or change-data-capture. Some have REST APIs; some have only web interfaces.

Procurement constraints. Modifying SWS requires engaging the original vendor (Microsoft, contracted at Rs 11.80 crore). Modifying any department system requires that department’s approval, budget, and a fresh procurement cycle. The SWS project was delayed 6 months due to integration complexity with just 30 departments. The pragmatic path is a solution that modifies *neither* side.

2.3 Scale of Impact

- **Businesses affected:** Karnataka processes over 2 lakh business registrations per annum through SWS, each touching 3-7 departments on average.
- **Data drift rate:** Based on the GST rollout experience (where state VAT and central GST portals diverged for months), we estimate 15-25% of cross-system records develop inconsistencies within 6 months.
- **Officer burden:** District-level officers currently rely on phone calls and WhatsApp to verify data across systems — an informal, unauditible reconciliation process.
- **Citizen impact:** Renewal applications rejected because department systems show stale addresses, causing 2-4 week delays for what should be automated approvals.

2.4 Lessons from Large-Scale Government IT

The GST rollout is the most instructive reference. When GSTN went live, state VAT systems were supposed to be subsumed. In practice, state systems ran in parallel for years, creating reconciliation nightmares. The lesson: **big-bang cutovers do not work in Indian government IT.** What works is a middleware layer that keeps parallel systems in sync until organic convergence happens — if it ever does.

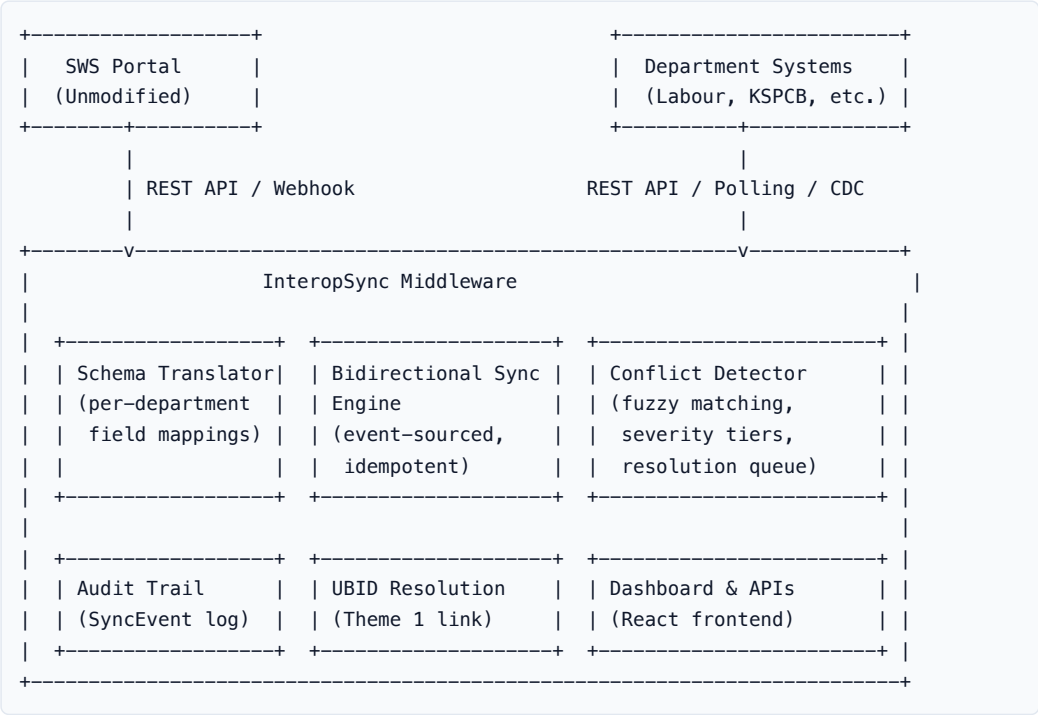
3. Solution Architecture

3.1 Design Principles

1. **Non-invasive:** The middleware sits alongside SWS and department systems. It reads from and writes to their existing interfaces. Neither SWS nor any department system requires modification.
2. **Bidirectional:** Changes can originate on either side. SWS-to-Department propagation handles new registrations and updates initiated on SWS. Department-to-SWS propagation handles status changes and approvals made by department officers.
3. **Event-sourced:** Every propagation is recorded as an immutable SyncEvent. The system can reconstruct the complete history of any business’s data across all systems.
4. **Idempotent:** Every sync operation computes a SHA-256 hash of the translated payload. If the exact same change has already been synced, it is silently skipped. This prevents duplicate writes when retrying failed operations.
5. **Conflict-aware:** When SWS and a department system disagree on a field value, the conflict is detected, classified by severity, and queued for human resolution. Data is never silently overwritten.

6. **UBID-anchored:** The Unified Business Identifier from Theme 1 is the join key across all systems. Without UBID, cross-system matching would require unreliable heuristics.

3.2 High-Level Architecture



3.3 Bidirectional Sync Engine

The sync engine is the core of the system. It handles two directions of propagation:

Direction 1 — SWS to Departments: 1. Receive SWS application payload (API trigger or webhook) 2. Use UBID to identify matching department records 3. Translate SWS schema to each department’s schema 4. Compute payload hash for idempotency 5. Detect conflicts with existing department data 6. If clean: write translated data, record COMPLETED SyncEvent 7. If conflicts: record CONFLICT SyncEvent, queue for resolution

Direction 2 — Departments to SWS: 1. Poll or snapshot-compare department records 2. Use UBID to find the corresponding SWS application 3. Translate department schema to SWS schema 4. Same idempotency check and conflict detection 5. Apply changes to SWS or queue conflicts

Both directions produce identical SyncEvent audit records, ensuring a unified trail regardless of propagation direction.

3.4 Schema Translation Layer

Each department has its own vocabulary. Our schema translator maintains explicit, configurable mappings for each department:

SWS Field	Labour	KSPCB	Commercial Tax	Factories	Fire Safet
business_name	establishment_name	unit_name	dealer_name	factory_name	building_r
owner_name	proprietor_name	contact_person	authorized_signatory	occupier_name	owner_oci
registered_address	premises_address	plant_address	business_address	factory_address	building_e
application_status	registration_status	consent_status	registration_status	license_status	noc_statu

Beyond field name translation, the system also handles **value translation**. SWS “approved” maps to Labour “Registered,” KSPCB “CTO Granted,” Commercial Tax “Active,” Factories “License Issued,” and Fire Safety “NOC Issued.” These mappings are stored as configuration, not code — adding a new department requires only a new mapping entry, not a code change.

Certain departments also require value transformations. For instance, KSPCB and Factories expect uppercase names. The translator applies these transformations automatically.

3.5 Conflict Detection and Resolution

Conflicts are detected by comparing SWS and department values for the same business, using the UBID as the join key. The system applies different comparison strategies based on field type:

- **Critical fields (status/registration):** Exact match required after normalisation. A mismatch between “approved” on SWS and “Registration Denied” on Labour is a critical conflict that must be resolved before sync can proceed.
- **Warning fields (names, addresses):** Fuzzy matching using token-sort ratio (via the rapidfuzz library). A similarity score below 80% for business names or 70% for addresses triggers a warning-level conflict. This accommodates minor spelling variations and transliteration differences common in Indian business names.
- **Info fields (pincode):** Exact match, but lower severity. A pincode mismatch is flagged but does not block sync.

Conflict resolution options: - **SWS Wins:** The SWS value is propagated to the department system. - **Department Wins:** The department value is propagated to SWS. - **Manual:** An officer enters a corrected value that is propagated to both systems. - **Merged:** A composite resolution (e.g., combining address details from both sources).

Every conflict is recorded with its detection timestamp, resolution timestamp, resolver notes, and chosen resolution — providing a complete decision audit trail.

3.6 Audit Trail

Every sync operation, whether successful or conflict-blocked, produces a `SyncEvent` record containing: - Direction (SWS-to-Dept or Dept-to-SWS) - Source and target system identifiers - The UBID linking both records - The original payload and the translated payload - A SHA-256 payload hash (for idempotency) - Status (pending, in_progress, completed, failed, conflict) - Attempt count and error messages (for retry tracking) - Timestamps for initiation and completion

This event log is queryable by UBID, department, direction, status, and time range — enabling officers to trace the complete synchronisation history of any business across all departments.

3.7 UBID as the Join Key

Without UBID, cross-system matching requires unreliable heuristics — fuzzy-matching business names, hoping PAN numbers are consistent, or manual linking by officers. Our Theme 1 implementation uses Splink (probabilistic record linkage) and IndicSoundex (phonetic matching for Indian names) to produce a stable UBID that persists across name changes, address changes, and status transitions. The middleware indexes UBID on every table and uses it as the primary join key.

This is not a hypothetical dependency. Both themes share the same monorepo, and UBID resolution is operational.

4. Government Feasibility and Deployment

4.1 The Non-Invasive Integration Strategy

The single most important characteristic of this solution is that it does not require any modification to SWS or to any department system. It integrates through three mechanisms, depending on what each system exposes:

1. **API-based integration:** For systems with REST APIs (SWS, some newer department portals), the middleware calls existing endpoints to read and write data.
2. **Webhook/event listeners:** For systems that emit events or notifications, the middleware subscribes to receive change alerts.
3. **Polling and snapshot comparison:** For legacy systems that expose neither APIs nor events, the middleware periodically reads current state and compares it to the last known snapshot to detect changes. This is the fallback for the majority of department systems.

4.2 Incremental Rollout Plan

Phase	Timeline	Scope	Departments
Pilot	Months 1-3	3 high-volume departments	Labour, Commercial Tax, KSPCB
Phase 2	Months 4-6	Extend to 5 more departments	Factories, Fire Safety, FSSAI, Drug Control, Town Planning
Phase 3	Months 7-12	All remaining departments	Full 40+ department coverage
Steady State	Month 12+	Maintenance, monitoring, new department onboarding	—

Each phase requires only: (a) schema mapping configuration for the new department, (b) integration adapter (API, webhook, or polling), and (c) status value translation table. No architectural changes are needed between phases.

4.3 Integration with Existing Infrastructure

- **NIC Data Centre / Karnataka State Data Centre:** Docker-containerised deployment on existing government infrastructure.
- **Aadhaar/DigiLocker:** UBID resolution can cross-reference DigiLocker-verified documents for higher confidence matching.
- **UMANG/mSeva:** Sync dashboards exposable through existing government super-app frameworks.
- **IndEA compliance:** Architecture follows the India Enterprise Architecture framework's principles for inter-system communication.

4.4 Cost Estimation

Component	Estimated Cost	Notes
Development (MVP to production)	Rs 25-35 lakhs	6-person team, 4 months
Infrastructure (annual)	Rs 8-12 lakhs	NIC Data Centre hosting, 3 servers
Department onboarding (per dept)	Rs 2-4 lakhs	Schema mapping, integration testing, UAT

Component	Estimated Cost	Notes
Maintenance (annual)	Rs 10-15 lakhs	2-person ops team, monitoring
Total Year 1	Rs 75 lakhs - 1.1 crore	Covers pilot + Phase 2

For context, the SWS itself was built for Rs 11.80 crore. This middleware is an order of magnitude cheaper because it does not build a new portal — it connects existing ones.

4.5 Change Management

The middleware introduces one new workflow: **conflict resolution**. When SWS and a department disagree, an officer chooses which value is authoritative. This is a workflow officers currently perform *informally* — by phoning each other or ignoring discrepancies. The middleware formalises it into a trackable, auditable process without adding net-new work.

Training requirements: - 2-hour orientation for department nodal officers - Self-service dashboard (no technical skills required) - Escalation path for unresolvable conflicts (to district-level coordinator)

5. Prototype and Demo

5.1 Working Prototype

Our prototype is a fully functional FastAPI backend with the following capabilities:

API Endpoints: - `POST /api/applications/` — Create an SWS application (simulates SWS submission) - `POST /api/sync/sws-to-dept` — Trigger SWS-to-Department sync for an application - `POST /api/sync/dept-to-sws` — Trigger Department-to-SWS sync for a department - `GET /api/sync/events` — Query the full audit trail with filtering by UBID, department, direction, and status - `GET /api/conflicts/` — List detected conflicts with severity and resolution status - `PUT /api/conflicts/{id}/resolve` — Resolve a conflict with SWS-wins, Dept-wins, Manual, or Merged strategy - `GET /api/conflicts/stats` — Aggregated conflict statistics by severity and department

Data Models: SWSApplication, DepartmentRecord, SyncEvent, SchemaMapping, ConflictRecord — fully relational with foreign keys and relationship mapping.

5.2 Demo Flow

The demonstration follows a realistic business lifecycle:

- Create application:** A business “Karnataka Textiles Pvt. Ltd.” submits a new registration on SWS with UBID `UBID-KA-2026-00042`.
- SWS-to-Department sync:** The middleware translates the SWS application into schemas for Labour, KSPCB, Commercial Tax, Factories, and Fire Safety. Each department receives the data in its own vocabulary (e.g., Labour sees “establishment_name: KARNATAKA TEXTILES PVT. LTD.”, KSPCB sees “unit_name: KARNATAKA TEXTILES PVT. LTD.”).
- Introduce a conflict:** The Labour Department updates the business name to “Karnataka Textile Industries Pvt Ltd” (note: “Textile” not “Textiles”, added “Industries”). Meanwhile, the business updates its address on SWS.
- Detect conflict:** When the next sync runs, the middleware detects:
 - Business name mismatch (fuzzy similarity < 80%) — severity: warning
 - Address change from SWS — no conflict (department has old value)

5. **Resolve conflict:** An officer reviews the conflict dashboard, sees both values side by side, and chooses "Manual" resolution with the corrected value "Karnataka Textile Industries Pvt. Ltd."
6. **Audit trail:** The complete history — original sync, conflict detection, resolution decision — is visible in the sync events log, filterable by UBID.

5.3 Dashboard

The conflict resolution dashboard provides: - Aggregate statistics: total conflicts, unresolved count, breakdown by severity and department - Filterable conflict list with SWS value vs department value side-by-side - One-click resolution with dropdown for resolution strategy - Sync event timeline for any UBID showing the complete propagation history

6. Scalability and Long-Term Impact

6.1 Horizontal Scaling Path

The event-sourced architecture is inherently scalable:

- **3 departments (pilot):** Single-instance deployment, SQLite database, synchronous processing. This is our current prototype.
- **40+ departments (Karnataka-wide):** PostgreSQL database, async task queue (Celery/Redis), horizontal scaling of sync workers. Each department's sync operations are independent and can run in parallel.
- **Multi-state deployment:** Each state gets its own middleware instance with state-specific schema mappings. A federation layer (inspired by ONDC's network architecture) can coordinate cross-state business registrations.

6.2 Production Enhancements

The prototype demonstrates the core sync logic. For production deployment, we would add:

- **Temporal.io** for durable workflow orchestration — ensures sync operations complete even if the middleware restarts mid-operation
- **Debezium** for change-data-capture on department databases that expose JDBC connections — eliminates polling overhead
- **Redis Streams** as the event bus between sync workers — provides backpressure and exactly-once delivery semantics
- **Prometheus/Grafana** for monitoring sync latency, conflict rates, and department-level health

6.3 Reference Architectures

Our design draws from proven federated interoperability patterns in Indian government IT:

- **India Stack:** Thin protocol layers (UPI, Aadhaar Auth, eSign) connecting thousands of heterogeneous systems without standardising the underlying systems.
- **ONDC:** Registry + protocol gateway enabling interoperability between diverse platforms — analogous to our middleware connecting SWS with diverse department systems.
- **Beckn Protocol:** Decentralised protocol for cross-network transactions. Our schema translation layer follows a similar normalisation-at-boundary pattern.
- **DIGIT (eGov Foundation):** Event-sourced architecture for cross-department data flow in urban governance — the closest architectural precedent to our design.

6.4 Long-Term Impact

If adopted statewide, InteropSync would: - **Eliminate data drift** for 2+ lakh business registrations per year across 40+ departments - **Reduce officer reconciliation time** from hours of phone calls to minutes of dashboard interaction - **Create a complete audit trail** of every cross-system data movement — currently nonexistent - **Enable data-driven policy:** Visibility into departmental bottlenecks, conflict patterns, and process improvement opportunities - **Pave the way for convergence:** As departments see consistent data flowing through the middleware, the case for maintaining separate systems weakens, enabling organic consolidation

7. Innovation Highlights

- 1. Middleware-as-reconciler, not middleware-as-router:** Existing integration approaches (ESB, API Gateway) route messages between systems. Our middleware actively reconciles divergent data — it detects when systems disagree and facilitates resolution, rather than blindly forwarding data.
- 2. Fuzzy conflict detection for Indian business data:** Indian business names involve transliterations, abbreviations, and spelling variants that exact matching cannot handle. Our use of rapidfuzz with calibrated thresholds (80% for names, 70% for addresses) accounts for this reality.
- 3. Cross-theme UBID integration:** The UBID is not a mock identifier — it is generated by our Theme 1 entity resolution engine using Splink probabilistic record linkage and IndicSoundex phonetic matching. This is the only submission (to our knowledge) that delivers a working UBID dependency.
- 4. Event-sourced audit trail:** Every propagation is an immutable event. The system can answer “what was the state of this business’s data across all departments on 15 March 2026?” — a capability that no current Karnataka system provides.
- 5. Per-department value semantics:** The schema translator handles not just field name differences but value-level semantics. SWS “approved” correctly becomes “Registered” for Labour, “CTO Granted” for KSPCB, “Active” for Commercial Tax, “License Issued” for Factories, and “NOC Issued” for Fire Safety. This domain awareness is critical for real-world accuracy.

8. Risks and Mitigations

Risk	Severity	Mitigation
Department systems lack APIs	High	Polling/snapshot comparison as fallback; screen-scraping adapter as last resort
Schema changes in department systems	Medium	Schema mappings are configuration, not code; version-tracked with rollback
High conflict volume overwhelms officers	Medium	Auto-resolution rules for low-severity conflicts; escalation thresholds
Network/infrastructure failures during sync	Medium	Idempotent operations + retry with exponential backoff; Temporal.io for durable execution
Resistance from department IT teams	Medium	Non-invasive integration; no changes to their systems; pilot with willing departments first
UBID resolution errors propagating bad links	Low	Confidence scoring on UBID matches; manual review queue for low-confidence links

Risk	Severity	Mitigation
Data privacy and access control	Medium	Role-based access; department officers see only their department's conflicts; audit logging of all access

9. Team and References

Team

- **Sridhar Suresh** — Full-stack engineer, backend architecture, sync engine design
- **Sruthi Krishnakumar** — Full-stack engineer, frontend, schema translation, conflict resolution UX

Technology Stack

- **Backend:** Python 3.12, FastAPI, SQLAlchemy, Pydantic
- **Conflict Detection:** rapidfuzz (fuzzy string matching)
- **Database:** SQLite (prototype), PostgreSQL (production)
- **Frontend:** React + TypeScript (Vite)
- **Infrastructure:** Docker, configurable for NIC Data Centre deployment

References

1. Karnataka Single Window System — Invest Karnataka portal (<https://investkarnataka.co.in>)
2. Microsoft contract for SWS development — Karnataka IT department procurement records, Rs 11.80 crore
3. GST Network rollout challenges — CAG Report on GST Implementation, 2019
4. India Stack — <https://indiastack.org>
5. ONDC Protocol Specification — <https://ondc.org>
6. Beckn Protocol — <https://becknprotocol.io>
7. Splink — Probabilistic record linkage at scale (<https://github.com/moj-analytical-services/splink>)
8. DIGIT Platform — eGov Foundation (<https://egov.org.in>)
9. Temporal.io — Durable execution framework (<https://temporal.io>)
10. Debezium — Change data capture (<https://debezium.io>)
11. IndEA — India Enterprise Architecture Framework, MeitY

This document accompanies a working prototype. All data used is synthetic — no real PII or government data is included.